Kubron

Инструкция пользователя и разработчика: модуль Helm

Версия 0.2.0 · Wails + Vue + Go · Октябрь 2026

Часть I. Инструкция пользователя

1. Обзор

Модуль Helm в Kubron даёт полноценное управление Helm-релизами прямо из desktop-клиента: просмотр, установка, обновление values и версии чарта, откат, удаление. Работа идёт через Kubernetes API (Secret/ConfigMap с релизами), без вызова `helm` CLI на машине пользователя.

Ключевые возможности:

- Список релизов с индикацией доступных обновлений чартов
- Каталог чартов: поиск, README, values, установка с dry-run
- Управление репозиториями (HTTP и OCI, basic auth, skip TLS)
- Upgrade values и смена версии чарта с предпросмотром diff манифеста
- История ревизий, diff, rollback
- Живой прогресс операций с возможностью отмены (install / upgrade)
- Журнал последних 200 операций с логом Helm
- Поддержка хранилища Secret (по умолчанию) и ConfigMap

ℹ️ Репозитории чартов хранятся локально в ~/.kubron/helm и не зависят от кластера. Релизы читаются из активного кластера.

2. Навигация по разделу Helm

В боковом меню доступны подразделы:

- Releases — установленные релизы в текущем (или выбранных) namespace
- Catalog — каталог чартов из добавленных репозиториев
- Repositories — управление источниками чартов
- Operations — история операций Helm

На дашборде кластера карточка Helm показывает сводку: число релизов, failed/pending, подсказки об обновлениях.

3. Репозитории (Helm → Repositories)

Список источников чартов хранится локально: `~/.kubron/helm`. Не привязан к кластеру.

3.1. Добавление репозитория

1. Нажмите «+ Добавить».
2. Укажите имя (например, bitnami) и URL.
3. При необходимости — логин/пароль (basic auth). Пароль сохраняется с правами 0600.
4. Для OCI-реестра укажите адрес вида oci://… и перечислите нужные чарты через запятую (у OCI нет индекса списка чартов).

5. При необходимости включите «Не проверять TLS-сертификат».
6. Нажмите «Добавить». Kubron проверит доступность и обновит индекс.

ℹ️ Незащищённый HTTP для OCI разрешён только для localhost.

3.2. Обновление индексов

- «🔄 Обновить все» — перезагрузить индексы всех репозиториев.
- У отдельного репозитория — кнопка «Обновить».

Для OCI версии берутся из тегов образов.

3.3. Удаление

Удаление репозитория не затрагивает уже установленные из него релизы.

4. Каталог чартов (Helm → Catalog)

Поиск по имени, описанию, ключевым словам. Фильтр по репозиторию. В таблице: чарт, репозиторий, версия, app-версия, описание. Помеченные как deprecated отображаются с меткой «устарел».

4.1. Карточка чарта

- README — описание чарта
- Values — values.yaml по умолчанию
- Info — сайт, источники, maintainers, keywords, версии
- Кнопка «Установить» открывает диалог установки

5. Установка релиза

Из каталога: выберите чарт → «Установить».

5.1. Параметры

- Имя релиза — обязательное, правила Kubernetes (строчные латинские буквы, цифры, «-», до 53 символов)
- Namespace — существующий или новый (можно создать)
- Версия чарта
- Хранилище релиза: Secret (по умолчанию, как helm CLI) или ConfigMap
- Дождаться готовности ресурсов (до 5 минут) — окно блокируется на время wait
- Values: слева — ваши переопределения, справа — defaults чарта (только для справки)

5.2. Предпросмотр (dry-run)

Кнопка «Предпросмотр» рендерит манифест без обращения к кластеру. Lookup-функции шаблонов возвращают пустое значение; значения Secret скрыты. Показываются Notes (NOTES.txt).

5.3. Выполнение

После «Установить» отображается живой прогресс. Операцию install можно отменить. По завершении релиз появится в Helm → Releases.

⚠️ Для установки нужны права на запись и (обычно) на secrets в целевом namespace.

6. Релизы (Helm → Releases)

Список последних ревизий каждого релиза. Колонки: имя, namespace, чарт, версия чарта, app-версия, ревизия, статус, обновлён, хранилище. Если в ваших репозиториях есть более новая версия чарта — показывается подсказка об обновлении.

6.1. Детали релиза

Откройте релиз — доступны вкладки:

- Content — user-values / all values (с defaults), Notes, копирование
- History — список ревизий, diff между ними, rollback
- Resources — ресурсы из манифеста релиза
- Upgrade (Edit values) — смена только values
- Chart version — смена версии чарта

7. История и откат

Вкладка History показывает все ревизии. Текущая помечена. Для любой пары ревизий можно посмотреть Diff (манифест / values).

Откат: выберите ревизию → «Откат». Создаётся новая ревизия с содержимым выбранной. Подтверждение обязательно.

ℹ️ Отмена (cancel) для rollback в текущей версии SDK Helm недоступна — операция дожидается завершения.

8. Обновление values (Edit values)

Меняются только values, версия чарта остаётся прежней.

- Слева — текущие user-values, справа — редактируемые
- Удалённый ключ возвращается к значению по умолчанию из чарта
- «Preview changes» — diff манифеста (Secret-значения скрыты)
- При устаревшей базе (релиз изменился пока вы редактировали) — предупреждение и кнопка перезагрузки values

После Apply создаётся новая ревизия. Есть живой прогресс и отмена.

9. Смена версии чарта (Chart version)

Переводит релиз на другую версию того же чарта. Требует прав на запись и доступ к секретам.

- Выберите репозиторий (если чарт найден в нескольких)
- Целевую версию (можно показать пререлизы)
- Метки: текущая / пререлиз / устарела
- Понижение версии помечается предупреждением: для безопасного возврата предпочтительнее rollback
- User-values переносятся; остальное берётся из defaults новой версии
- Предпросмотр diff манифеста, затем «Обновить чарт»

ℹ️ Если чарт не найден ни в одном репозитории — добавьте репозиторий в Helm → Repositories и обновите индекс.

10. Удаление релиза

Из списка релизов или деталей — действие Uninstall. Подтверждение. Операция выполняется с живым прогрессом (cancel для uninstall в текущем SDK недоступен).

11. Журнал операций (Helm → Operations)

История установок, обновлений values, смен версии чарта, откатов и удалений. Хранятся последние 200 операций. У каждой — журнал вывода Helm.

- Фильтры: тип операции, статус (успех / ошибка / отменено), поиск по релизу, namespace, кластеру, тексту ошибки
- Выполняющиеся операции выделены отдельно
- Очистка истории не затрагивает audit log

В web-режиме операция привязана к сессии владельца: отменить может только он.

12. Права и роли

Локальные роли Kubron:

Роль	Доступ к Helm
admin	Полный: install, upgrade, chart upgrade, rollback, uninstall, репозитории
operator	Операции с релизами (write / secrets); управление репозиториями — по политике
user	Просмотр релизов, истории, каталога (read)

Дополнительно действуют RBAC кластера: без прав на secrets/configmaps в namespace список релизов будет неполным или пустым.

13. Хранилище релизов

Helm по умолчанию пишет ревизии в Secret типа `helm.sh/release.v1` (label owner=helm). Альтернатива — ConfigMap (HELM_DRIVER=configmap). Kubron читает оба хранилища через API и при установке позволяет выбрать тип.

Данные релиза: base64(gzip(JSON)). Superseded-ревизии не показываются в основном списке, но доступны в History.

14. Прогресс и отмена

Для install и upgrade (values / chart version) отображается панель прогресса с логом. Можно отменить операцию. Rollback и uninstall в текущей интеграции с Helm SDK cancel не поддерживают — дожидаются завершения.

15. Практические советы

7. Сначала добавьте нужные репозитории (bitnami, prometheus-community, ваши OCI) и обновите индексы.
8. Перед установкой в prod используйте dry-run и внимательно смотрите Notes.
9. Для безопасного отката после неудачного upgrade предпочитайте History → Rollback, а не повторный chart-downgrade.

10. Если релиз «пропал» — проверьте namespace-фильтр и права на secrets/configmaps.
11. Индикатор обновления чарта появляется только если соответствующий репозиторий добавлен и индекс актуален.
12. В multi-cluster режиме репозитории общие (локальные), релизы — в контексте активного кластера.

16. Ограничения текущей версии (0.2.0)

- Cancel доступен только для install и upgrade
- Плагины Helm и post-renderer пока не поддерживаются
- OCI: нужно явно перечислять имена чартов
- Web-режим: UI Helm подключён, но web-версия в целом ещё в активной разработке
- Нет встроенного lint/template validate сверх dry-run install

Часть II. Документация для разработчиков

17. Архитектура модуля

Helm в Kubron — не обёртка над CLI, а интеграция через `helm.sh/helm/v4` SDK + прямое чтение Secret/ConfigMap через client-go.

17.1. Слои

Слой	Ответственность
Frontend (Vue 3)	Views, tabs, composables, resource definitions, i18n
Wails bridge	callApp('MethodName', ...) → Go Manager methods
internal/k8s (Go)	Helm SDK actions, store, repo, progress, journal
Kubernetes API	Secrets/ConfigMaps (релизы), REST для apply
Локальный FS	~/.kubron/helm — repos, cache, operations.json

17.2. Backend-файлы (internal/k8s/)

Файл	Назначение
helm.go	GetHelmReleases / GetHelmCharts, list payloads
helm_store.go	Чтение Secret + ConfigMap, decode base64/gzip/JSON
helm_actions.go	Rollback, Uninstall, Diff, helmConfig*
helm_install.go	Install + dry-run, HelmInstallRequest/Result
helm_upgrade.go	Upgrade values only
helm_chart_upgrade.go	Смена версии чарта (v0.2.0)
helm_detail.go	Content (values/notes/manifest) по ревизии
helm_repo.go	Репозитории HTTP/OCI, index, search, package cache
helm_oci.go	OCI registry: теги, pull chart
helm_updates.go	Поиск новее установленной версии в репо
helm_progress.go	opID, tracker, cancel, событие helm:progress
helm_journal.go	operations.json, последние 200 записей
helm_cache.go	Informer / событие helm:changed

Каждый файл имеет `*_test.go`.

17.3. Frontend

- `frontend/src/components/views/` — HelmReposView, HelmCatalogView, HelmInstallDialog, HelmOperationsView
- `frontend/src/components/resource/sections/` — HelmContentTab, HelmHistoryTab, HelmResourcesTab, HelmUpgradeTab, HelmChartUpgradeTab
- `frontend/src/composables/useHelmOps.ts` — прогресс, cancel, runHelmOp, helmJournalTick
- `frontend/src/composables/useHelmLive.ts` — helmLiveTick по событию helm:changed
- `frontend/src/resources/definitions/helmrelease.ts` — defineResource: columns, tabs, actions (uninstall)
- `frontend/src/resources/definitions/helmchart.ts` — агрегация по chart
- `frontend/src/i18n/locales/{en,ru}/helm.json` — все строки UI

18. Модель данных (Go)

HelmReleaseInfo

Последняя ревизия релиза для списка: name, namespace, chart, chartVersion, appVersion, revision, status, updated, description, storage (secret|configmap), latestVersion / latestRepo / latestDeprecated.

HelmInstallRequest

repo, chart, version, releaseName, namespace, values (YAML overrides), createNamespace, wait, storage, opId.

HelmInstallResult

releaseName, namespace, revision, status, manifest (secrets masked), notes.

HelmRevisionDiff

from, to, values (unified diff), manifest (unified diff, secrets masked).

HelmOperationRecord

id, op, cluster, namespace, release, detail, revision, status (success|failed|canceled), error, startedAt, finishedAt, durationMs, log[].

HelmRepoInfo

name, url, username, insecureTLS, charts, updated, error, kind (http|oci), chartNames (для OCI).

19. Хранилище релизов (helm_store.go)

Формат объекта: `sh.helm.release.v1.<имя>.v<N>`, label `owner=helm`, поле `data.release = base64(gzip(JSON))`.

- Тип Secret: helm.sh/release.v1
- ConfigMap: нет type, признак — поле release + label
- listHelmStored читает оба; отказ в доступе к одному хранилищу терпим, если второе доступно
- decode: base64 → gzip (LimitReader 64 MiB) → JSON
- Операции (rollback/upgrade/uninstall) идут через тот же driver, где лежит релиз — иначе Helm его не найдёт

Константы: `helmStorageSecret` = "secret", `helmStorageConfigMap` = "configmap".

20. Репозитории (helm_repo.go)

Каталог: `~/.kubron/helm/` (не ~/.config/helm).

- `repositories.yaml` — формат Helm repo.File, mode 0600 (там пароли)
- `cache/<name>-index.yaml` — скачанные индексы
- Имя репозитория валидируется строго (regex), входит в имя файла
- Credentials уходят только на хост репозитория (не на redirect / chart URL из index)
- Лимиты: index 64 MiB, chart archive 32 MiB, HTTP timeout 30s, parallel update 4
- In-memory cache индексов и LRU package cache (16)

OCI: отдельный путь (helm_oci.go) — нет index.yaml, версии = теги, chart names задаёт пользователь.

21. Операции и прогресс (helm_progress.go)

Схема:

13. Фронт генерирует opId (UUID / [A-Za-z0-9-]{8,64}) до вызова backend.
14. Backend: beginHelmOp(ctx, opID, op, ns, rel, cancelable) → tracker + (опционально) cancel context.
15. Событие
16. Batch строк журнала ≤ 40, tick 250 ms, heartbeat 1 s.
17. CancelHelmOperation(opId) вызывает tracker.cancel (только если cancelable=true).
18. finish() пишет запись в journal и шлёт done=true.

Cancelable: install и upgrade — да (контекст Helm action). Rollback и uninstall — нет (Helm SDK не принимает context для этих action в текущей интеграции); timeout 5 минут.

Frontend (useHelmOps.ts):

- Подписка onEvent('helm:progress')
- helmOps ref — активные + недавно завершённые
- Успешные auto-dismiss через 6 s
- helmJournalTick++ при done → страница Operations перечитывает историю
- runHelmOp({ op, namespace, release, cancelable }, fn) — обёртка

ℹ️ В web-режиме операция привязана к сессии владельца: отменить может только владелец.

22. Live-обновления (helm_cache.go + useHelmLive)

Informer на Secret (и при возможности ConfigMap) с label owner=helm. При изменении — EventsEmit `helm:changed`.

useHelmLive: helmLiveTick++. Resource definition helmrelease подписан через refreshOn / autoRefresh. Если у пользователя нет list secrets — informer не стартует, остаётся polling.

23. Журнал операций (helm_journal.go)

Файл: `~/.kubron/helm/operations.json` (0600), общий для всех кластеров; в записи есть cluster.

- Максимум 200 операций, у каждой — до 80 строк лога
- Это operational log, не audit: «кто» пишет audit log
- GetHelmOperations(query), GetHelmOperation(id), ClearHelmOperations()

24. Resource definition (frontend)

helmrelease: defineResource с namespaced: false (агрегат), editable: false (нет YAML-редактора объекта).

Tabs: values / notes / manifest (helmContentTab), editValues (HelmUpgradeTab), upgradeChart (HelmChartUpgradeTab), resources, history.

Actions: uninstall (Perm.Write, danger, confirm → runHelmOp).

Колонка update: latestVersion из backend (helm_updates.go + локальные индексы репо).

25. Права и безопасность

- Локальные Perm (Read / Write / Secrets / …) проверяются на UI и на backend
- GetHelmReleaseContent / Diff требуют Secrets — в values могут быть пароли
- Маскирование Secret в manifest/diff перед отдачей на UI
- Пароли репозиториев: 0600, не логируются, уходят только на host репо
- validateHelmRepoName / validateHelmRef / chartutil.ValidateReleaseName — до сети
- Gzip bomb protection: LimitReader 64 MiB при decode

26. Зависимости

- `helm.sh/helm/v4` — action, chart, repo, kube, release
- `k8s.io/client-go` / cli-runtime — REST, ConfigFlags, informers
- Wails EventsEmit / EventsOn для progress и live

Helm v4: action.Configuration.Init(flags, namespace, driver), WaitStrategy = kube.StatusWatcherStrategy.

27. Как добавить новую Helm-операцию

19. Backend: метод на Manager, beginHelmOp(…, cancelable), defer tr.finish(err), использовать helmConfigForRelease / helmConfigDriver.
20. Если нужен progress-лог — передать tr.handler() в SetLogger до Init.
21. Замаскировать secrets в любом возвращаемом манифесте.
22. Frontend: runHelmOp({ op, namespace, release, cancelable }, () => callApp('Method', …)).
23. Добавить i18n-ключи в en/ru helm.json (ops_panel, operations).
24. Тест: *_test.go с fake client / httptest для repo.

28. Тесты

Все helm_*.go имеют table-driven тесты. Типичные кейсы:

- decode повреждённого / oversized release payload
- list при Forbidden на secrets, но OK на configmaps (и наоборот)
- rollback/upgrade с неверным storage driver
- repo name validation, index size limits
- opID validation, cancel race
- journal trim до 200

29. Связанные разделы продукта

- Dashboard — карточка Helm (HelmCard.vue)
- Argo CD — отдельный модуль (GitOps), не смешивать с Helm releases
- Audit log — кто запускал операции (локальные пользователи)
- pkg/ext — точки расширения (SSO и т.п.), не Helm-specific

Документ составлен по коду и UI Kubron v0.2.0 (https://github.com/VadimII224/kubron). При расхождениях с кодом приоритет у репозитория.